CredShields

Smart Contract Audit

June 26, 2026 • CONFIDENTIAL

Description

This document details the process and result of the Smart Contract audit performed by CredShields Technologies PTE. LTD. on behalf of Sivo between June 5, 2026, and June 10, 2026. A retest was performed on June 26, 2026.

Author

Shashank (Co-founder, CredShields) shashank@CredShields.com

Reviewers

Aditya Dixit (Research Team Lead), Shreyas Koli (Auditor), Prasad Kuri (Auditor), Neel Shah (Auditor)

Prepared for

Sivo

Table of Contents

1. Executive Summary ----------

Sivo engaged CredShields to perform a smart contract audit from June 5, 2026, to June 10, 2026. During this timeframe, 13 vulnerabilities were identified. **A retest was performed on June 26, 2026, and all the bugs have been addressed.**

During the audit, 0 vulnerabilities were found with a severity rating of either High or Critical. These vulnerabilities represent the greatest immediate risk to "Sivo" and should be prioritized for remediation, and fortunately, none were found.

The table below shows the in-scope assets and a breakdown of findings by severity per asset. Section 2.3 contains more information on how severity is calculated.

Assets in Scope	Critical	High	Medium	Low	info	Gas	Σ
Vault Contract	0	0	5	1	7	0	**13**

Table: Vulnerabilities Per Asset in Scope

The CredShields team conducted the security audit to focus on identifying vulnerabilities in Vault Contract's scope during the testing window while abiding by the policies set forth by Sivo's team.

State of Security

To maintain a robust security posture, it is essential to continuously review and improve upon current security processes. Utilizing CredShields' continuous audit feature allows both Sivo's internal security and development teams to not only identify specific vulnerabilities but also gain a deeper understanding of the current security threat landscape.

To ensure that vulnerabilities are not introduced when new features are added, or code is refactored, we recommend conducting regular security assessments. Additionally, by analyzing the root cause of resolved vulnerabilities, the internal teams at Sivo can implement both manual and automated procedures to eliminate entire classes of vulnerabilities in the future. By taking a proactive approach, Sivo can future-proof its security posture and protect its assets.

2. The Methodology ----------

Sivo engaged CredShields to perform a Vault Contract audit. The following sections cover how the engagement was put together and executed.

2.1 Preparation Phase

The CredShields team meticulously reviewed all provided documents and comments in the smart contract code to gain a thorough understanding of the contract's features and functionalities. They meticulously examined all functions and created a mind map to systematically identify potential security vulnerabilities, prioritizing those that were more critical and business-sensitive for the refactored code. To confirm their findings, the team deployed a self-hosted version of the smart contract and performed verifications and validations during the audit phase.

A testing window from June 5, 2026, to June 10, 2026, was agreed upon during the preparation phase.

2.1.1 Scope

During the preparation phase, the following scope for the engagement was agreed upon:

IN SCOPE ASSETS
• **Audited Scope:** https://etherscan.io/address/0x77ea87319f48c584f0c444659d397adc2278101c#code • **Retested Scope:** https://etherscan.io/address/0x55ff697469f0AD9762636c7f5acb9b689C6314cA#code

2.1.2 Documentation

Documentation was not required as the code was self-sufficient for understanding the project.

2.1.3 Audit Goals

CredShields employs a combination of in-house tools and thorough manual review processes to deliver comprehensive smart contract security audits. The majority of the audit involves manual inspection of the contract's source code, guided by OWASP's Smart Contract Security Weakness Enumeration (SCWE) framework and an extended, self-developed checklist built from industry best practices. The team focuses on deeply understanding the contract's core logic, designing targeted test cases, and assessing business logic for potential vulnerabilities across OWASP's identified weakness classes.

CredShields aligns its auditing methodology with the OWASP Smart Contract Security projects, including the Smart Contract Security Verification Standard (SCSVS), the Smart Contract Weakness Enumeration (SCWE), and the Smart Contract Secure Testing Guide (SCSTG). These frameworks, actively contributed to and co-developed by the CredShields team, aim to bring consistency, clarity, and depth to smart contract security assessments. By adhering to these OWASP standards, we ensure that each audit is performed against a transparent, community-driven, and technically robust baseline. This approach enables us to deliver structured, high-quality audits that address both common and complex smart contract vulnerabilities systematically.

2.2 Retesting Phase

Sivo is actively partnering with CredShields to validate the remediations implemented towards the discovered vulnerabilities.

2.3 Vulnerability classification and severity

CredShields follows OWASP's Risk Rating Methodology to determine the risk associated with discovered vulnerabilities. This approach considers two factors - Likelihood and Impact - which are evaluated with three possible values - **Low**, **Medium**, and **High**, based on factors such as Threat

agents, Vulnerability factors, and Technical and Business Impacts. The overall severity of the risk is calculated by combining the likelihood and impact estimates.

Overall Risk Severity				
Impact	HIGH	Medium	High	Critical
	MEDIUM	Low	Medium	High
	LOW	None	Low	Medium
		LOW	MEDIUM	HIGH
Likelihood				

Overall, the categories can be defined as described below -

1. **Informational**

   We prioritize technical excellence and pay attention to detail in our coding practices. Our guidelines, standards, and best practices help ensure software stability and reliability. Informational vulnerabilities are opportunities for improvement and do not pose a direct risk to the contract. Code maintainers should use their own judgment on whether to address them.

2. **Low**

   Low-risk vulnerabilities are those that either have a small impact or can't be exploited repeatedly or those the client considers insignificant based on their specific business circumstances.

3. **Medium**

   Medium-severity vulnerabilities are those caused by weak or flawed logic in the code and can lead to exfiltration or modification of private user information. These vulnerabilities

can harm the client's reputation under certain conditions and should be fixed within a specified timeframe.

4. **High**

   High-severity vulnerabilities pose a significant risk to the Smart Contract and the organization. They can result in the loss of funds for some users, may or may not require specific conditions, and are more complex to exploit. These vulnerabilities can harm the client's reputation and should be fixed immediately.

5. **Critical**

   Critical issues are directly exploitable bugs or security vulnerabilities that do not require specific conditions. They often result in the loss of funds and Ether from Smart Contracts or users and put sensitive user information at risk of compromise or modification. The client's reputation and financial stability will be severely impacted if these issues are not addressed immediately.

6. **Gas**

   To address the risk and volatility of smart contracts and the use of gas as a method of payment, CredShields has introduced a "Gas" severity category. This category deals with optimizing code and refactoring to conserve gas.

2.4 CredShields staff

The following individual at CredShields managed this engagement and produced this report:

- Shashank, Co-founder CredShields shashank@CredShields.com

Please feel free to contact this individual with any questions or concerns you have about the engagement or this document.

3. Findings Summary ----------

This chapter contains the results of the security assessment. Findings are sorted by their severity and grouped by asset and OWASP SCWE classification. Each asset section includes a summary highlighting the key risks and observations. The table in the executive summary presents the total number of identified security vulnerabilities per asset, categorized by risk severity based on the OWASP Smart Contract Security Weakness Enumeration framework.

3.1 Findings Overview

3.1.1 Vulnerability Summary

During the security assessment, 13 security vulnerabilities were identified in the asset.

Vulnerability Title	Severity	Vulnerability Type	Status
Cross-asset arbitrage via shared pooled accounting	Medium	Business Logic (SC02-LogicErrors)	Acknowledged
maxWithdraw / maxRedeem over-report free liquidity, enabling settled-claim drain or revert	Medium	Business Logic (SC02-LogicErrors)	Fixed
Oracle/pause gate freezes already-settled (claimable) redemptions	Medium	Denial of Service (SCWE-087)	Fixed
Asynchronous lifecycle and RWA operations bypass both the pause and the oracle depeg gate	Medium	Access Control (SCWE-016)	Fixed
deploy / maxDeploy use the raw balance, making escrow, reserves, and fees deployable to RWA	Medium	Business Logic (SC02-LogicErrors)	Fixed
Reentrancy in request-deposit / request-redeem enables a double-credit drain with hooked tokens	Low	Reentrancy Attacks (SCWE-046)	Fixed
redeem / withdraw above a claimable bucket underflow-reverts	Informational	Business Logic (SC02-LogicErrors)	Acknowledged

pause() reverts during an oracle anomaly	Informational	Denial of Service (SCWE-087)	Fixed
A reverting or non-conforming oracle bricks the paused() view and every gated path	Informational	Denial of Service (SCWE-087)	Fixed
Missing zero-amount guards on the withdraw and asynchronous-settlement paths	Informational	Lack of Input Validation (SC05-Lack Of Input Validation)	Fixed
Oracle feed decimals are not validated against the configured price band	Informational	Lack of Input Validation (SC05-Lack Of Input Validation)	Fixed
Heterogeneous-decimal assets on one share corrupt the shared pool and enable a cross-vault drain	Informational	Business Logic (SC02-LogicErrors)	Fixed
Direct asset donations are permanently stuck	Informational	Locked Funds	Fixed

Table: Findings and Remediations

5. Bug Reports ----------

Bug ID #M001 [Acknowledged]

Cross-asset arbitrage via shared pooled accounting

Vulnerability Type
Business Logic (SC02-LogicErrors)

Severity
Medium

Description
All three per-asset vaults share one fungible share token and price it from a single global totalAssets()/totalSupply() that treats every asset as 1:1, while each vault physically redeems only its own token. An attacker can deposit the cheapest or depegged stablecoin into one vault, mint shares at the global $1-equivalent NAV, and redeem a stronger asset from another vault, extracting the peg spread from that vault's depositors. The deposit-side oracle is the only gate, but the USD5 vault has no oracle configured (oracle == address(0), which fails open) and the instant-redeem fee is zero, so the weak-leg deposit is ungated and the round-trip is fee-free. Only the current fully-deployed state (zero idle liquidity) prevents immediate execution.

Affected Code

- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F40#L220
- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F46#L169

Impacts
During any stablecoin depeg, an arbitrageur can convert the weak/depegged asset into the stronger asset at par, draining the higher-value vaults of their liquidity at the expense of honest depositors.

Remediation
Segregate accounting per asset, a dedicated share token per asset, or per-asset idle and deployed sub-pools that each vault redeems strictly against its own asset and price every vault through its own depeg oracle. At a minimum, configure a depeg oracle on every vault

Retest

Acknowledged, the shared-pool design is unchanged. The synchronous deposit/mint paths are now oracle-gated, and the client risk-accepts the residual under the operator-trust model. Acceptance is conditional on: (1) USD5 being non-market / mint-only-on-fiat and documented as such; (2) each market vault's live oracle band being tight enough to trip a real depeg; and (3) the operator never settling a depegged request the asynchronous deposit/settlement remains ungated (see M004, Not Fixed). The instant-redeem fee is secondary only: it does not stop a real depeg and is bypassed by the no-fee asynchronous redeem path.

Root cause and path to a full fix: M001 traces to one design choice a single fungible sivoUSDX share shared across all vaults, so shares minted by depositing one asset can be redeemed for another (deposit USDC → withdraw USDT). If isolated per-asset vaults fit the intended design, bind the asset to the share itself a separate share and separate accounting per asset (sivoUSDC/sivoUSDT, each still transferable and yield-bearing, but redeemable only for its own asset); applying this fixes M001 at the root.

Bug ID #M002 [Fixed]

maxWithdraw / maxRedeem over-report free liquidity, enabling settled-claim drain or revert

Vulnerability Type
Business Logic (SC02-LogicErrors)

Severity
Medium

Description
maxRedeem and maxWithdraw cap on the vault's raw physical token balance, but the spend is gated by the share burn, which decrements the global shared idle accounting and then transfers assets out of this vault. The raw balance also holds pending-deposit escrow, claimable-redeem reserves, and accumulated fees, none of which are free for a redeemer, so the cap over-reports. Depending on the relation between the resolved amount, this vault's own idle, and the global idle, the result is either an underflow revert when global idle is short (breaking the requirement that the max views must not cause a revert, and griefable by front-running a later-cancelled deposit that inflates the raw balance), or a successful burn that pays the redeemer out of this vault's earmarked escrow, fees, and settled-redemption reserves, leaving those claimants unpayable until an operator recall.

Affected Code

- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F46#L266
- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F46#L283
- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F46#L291

Impacts
A redeemer can drain another party's earmarked escrow, accrued fees, or settled-redemption reserves and freeze those claims, or deterministically revert redemptions sized off the reported maximum, breaking integrators.

Remediation
Bound the liquidity caps on a free, unencumbered balance the physical balance minus pending-deposit escrow, claimable-redeem reserves, and accumulated fees.

Retest

maxWithdraw and maxRedeem are now bound on unencumbered liquidity instead of the raw physical-balance.

Bug ID #M003 [Fixed]

Oracle/pause gate freezes already-settled (claimable) redemptions

Vulnerability Type
Denial of Service (SCWE-087)

Severity
Medium

Description
The paused state folds oracle health into itself, returning paused whenever the feed is stale or out of band. A settled redemption is already finalized: the shares have been burned and the assets reserved at a price locked at settlement, so claiming those reserved assets needs no oracle and no conversion, only a transfer. Yet the only claim path is withdraw/redeem, which is gated by the not-paused modifier. A depeg or feed outage therefore freezes a finalized, price-locked obligation exactly when the user most needs to exit, with no recourse the cancel path only touches the already-empty pending bucket, and unpausing cannot clear the oracle term. The mirror deposit-claim path is not pause-gated and keeps working, an inconsistency.

Affected Code

- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F46#L97
- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F38#L288
- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F46#L107

Impacts
Users with already-settled, self-funded redemptions are blocked from claiming their reserved assets during exactly the depeg or outage when they most need to exit.

Remediation
Provide a dedicated claim function for settled redemptions that is not gated by oracle health and pays out only the caller's reserved bucket, while keeping the live-NAV instant-redeem path oracle-gated; and separate the manual emergency pause from the oracle-health gate.

Retest
Oracle health is decoupled from the manual pause. paused() now reflects only the manual emergency pause (PAUSER_ROLE); the oracle gate is a separate predicate.

Bug ID #M004 [Fixed]

Asynchronous lifecycle and RWA operations bypass both the pause and the oracle depeg gate

Vulnerability Type
Access Control (SCWE-016)

Severity
Medium

Description
Only the four synchronous entry points carry the not-paused modifier. The entire asynchronous lifecycle (request-deposit, request-redeem, the cancel functions, and the claim functions) and every operator operation (deploy, recall, settlement, fee claim) have only a role gate. Two consequences follow. The manual pause is an incomplete emergency stop, because users can still queue requests and the operator can still settle, deploy, and recall while paused. And the oracle depeg gate misses the path that actually mints shares at a price: the deposit settlement mints at the pooled par NAV, so during a depeg a user can request a deposit of the depegged asset and have it settled at the wrong price, executing the weak leg of the cross-asset arbitrage through the ungated asynchronous door while the breaker sits only on the synchronous door.

Affected Code

- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F38#L111
- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F46#L433

Impacts
The emergency pause does not actually stop the protocol, and the depeg breaker does not protect the settlement path that mints at NAV, allowing minting at a wrong price during a depeg.

Remediation
Apply the pause and oracle gate to the deposit-in and settlement path (request-deposit and the settlement that mints at NAV), while leaving user-exit paths and operator recall ungated so users can always exit and liquidity can always be restored.

Retest
Fixed. requestDeposit now carries the whenNotPaused modifier, and claimableDeposit reverts when assets > maxDeposit(controller). Because maxDeposit() returns 0 whenever

_liveTradingHalted() (paused() or price out of range), the mint-at-NAV settlement is now blocked during both a pause and an oracle depeg, closing the ungated path that previously let a depegged deposit settle at the wrong price.

Bug ID #M005 [Fixed]

deploy / maxDeploy use the raw balance, making escrow, reserves, and fees deployable to RWA

Vulnerability Type
Business Logic (SC02-LogicErrors)

Severity
Medium

Description
maxDeploy caps by the minimum of the global idle-based budget and the vault's raw physical balance. Because the physical cap counts escrow, reserves, and fees, and the budget is global across all vaults, a vault can deploy more than its own settled idle, physically sending pending-deposit escrow and claimable-redeem reserves out to the operator. In the shared multi-vault configuration the global idle can exceed a single vault's raw balance, so the raw balance becomes the binding cap and a full deploy exports the entire physical balance, escrow and fees included. This breaks cancel-deposit, redeem-claims, and fee claims for the affected parties; in the worst case it also cuts the global idle below the physical idle held in sibling vaults (throttling their redemptions) and counts the swept escrow as deployed principal that then accrues phantom yield.

Affected Code

- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F46#L402
- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F46#L413

Impacts
Pending-deposit escrow, settled-redemption reserves, and accrued fees can be physically sent to the operator, freezing those claims and distorting the global accounting.

Remediation
Bound deploy and maxDeploy by the vault's own free, unencumbered balance its idle minus its earmarked escrow, fees, and reserves independent of the global budget, so a vault can never export more than its own free liquidity.

Retest
Both deploy paths are now bounded by the vault's own free balance.

Bug ID #L001 [Fixed]

Reentrancy in request-deposit / request-redeem enables a double-credit drain with hooked tokens

Vulnerability Type
Reentrancy Attacks (SCWE-046)

Severity
Low

Description
Request-deposit credits a balance-delta measured around the token transfer, with no reentrancy guard; the synchronous deposit path is guarded, but the asynchronous path is not. With a token that has transfer hooks, the sender's pre-transfer hook fires before settlement; a reentrant inner request-deposit credits its own delta, and then the outer delta (balance minus the earlier snapshot) captures both transfers, over-crediting the pending amount and enabling a drain through cancel or claim. The store explicitly advertises hooked-token support. The three currently supported assets are not hooked, so the path is presently unreachable, but it becomes exploitable the moment a hooked asset is onboarded.

Affected Code

- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F38#L118
- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F38#L120

Impacts
With a hooked asset, an attacker can over-credit their pending deposit and withdraw more than they deposited, draining the vault.

Remediation
Add a reentrancy guard to the asynchronous request and cancel functions.

Retest
requestDeposit, requestRedeem, cancelDeposit, and cancelRedeem now carry the nonReentrant-modifier

Bug ID #I001 [Acknowledged]

redeem / withdraw above a claimable bucket underflow-reverts

Vulnerability Type
Business Logic (SC02-LogicErrors)

Severity
Informational

Description
When a controller's claimable-redeem bucket is non-empty, withdraw and redeem pay the entire requested amount from the bucket only, with no fallthrough to the synchronous instant path. Any request exceeding the bucket underflow-reverts, whether the over-amount comes from the maximum sentinel (which keys off the wallet balance and is blind to the bucket) or from an explicit number. The position remains recoverable in a separate transaction drain the bucket first, then redeem the wallet shares so no value is lost, but a call sized to the reported maximum can revert, which can break integrators that rely on that value.

Affected Code

- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F38#L265
- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F38#L288

Impacts
A redemption sized to the reported maximum can revert for a holder who has both a settled bucket and additional wallet shares, breaking automated integrations; the funds remain recoverable manually.

Remediation
Let a single call claim the bucket and route any remainder through the synchronous path, or make the redeemable-amount view account for the claimable bucket.

Retest
Behaviour is unchanged: when a controller's claimable-redeem bucket is non-empty, withdraw/redeem pay strictly from that bucket and an over-amount underflow-reverts, with no fallthrough to the synchronous instant path.

Bug ID #I002 [Fixed]

pause() reverts during an oracle anomaly

Vulnerability Type
Denial of Service (SCWE-087)

Severity
Informational

Description
The manual pause function inherits the not-paused modifier, and the not-paused check is overridden to revert on oracle anomalies, so the pauser cannot set a manual pause while the feed is stale or out of range. During such an anomaly the vault is already auto-paused, so the only practical effect is that a manual pause cannot be latched to persist after the feed recovers the vault silently auto-unpauses.

Affected Code

- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F46#L255

Impacts
The pauser cannot establish a manual pause that survives an oracle recovery; there is no fund impact.

Remediation
Make the manual pause path independent of the oracle gate, applying the oracle check only to the user entry points.

Retest
pause() is now onlyRole(PAUSER_ROLE) calling _pause() directly.

Bug ID #I003 [Fixed]

A reverting or non-conforming oracle bricks the paused() view and every gated path

Vulnerability Type
Denial of Service (SCWE-087)

Severity
Informational

Description
The price check calls the oracle's latest-round read without a try/catch, and the staleness subtraction is unguarded. A feed that reverts (for example a deprecated or sunset feed) or returns a future timestamp bubbles a revert through the paused view, which is called by the maximum views and by every gated write path, bricking the synchronous paths until the manager swaps the oracle.

Affected Code

- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F46#L172

Impacts
A misbehaving or deprecated feed temporarily bricks deposits and redemptions until an admin reconfigures the oracle.

Remediation
Wrap the oracle read in a try/catch and treat a failure as an incomplete round, and guard the staleness subtraction against a timestamp in the future.

Retest
The oracle read is wrapped in try/catch.

Bug ID #I004 [Fixed]

Missing zero-amount guards on the withdraw and asynchronous-settlement paths

Vulnerability Type
Lack of Input Validation (SC05-Lack Of Input Validation)

Severity
Informational

Description
The synchronous deposit path rejects a degenerate zero-share mint, but the symmetric guard is missing on three sibling paths. The synchronous withdraw has no zero-asset guard, so with a non-zero fee or a large price drift a user can burn shares for zero assets, the whole amount captured as fee. The deposit settlement has no zero-share guard, so a dust deposit can mint zero shares while still crediting idle, silently donating the dust to existing holders. The redeem settlement has no zero-asset guard, so at a sub-unit valuation it can burn the requester's shares for zero reserved assets. All are dust-level and dormant under the current zero-fee, equal-decimal, above-par configuration.

Affected Code

- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F40#L276
- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F38#L240
- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F38#L147

Impacts
Under a non-zero fee or a sub-par price, dust-sized operations can burn shares for nothing or donate value to other holders; the amounts involved are negligible.

Remediation
Mirror the deposit zero-share guard on the withdraw path and on both settlement paths, reverting or skipping when the minted shares or the paid assets round to zero.

Retest
Zero-amount guards were added on all three previously-unguarded sibling paths.

Bug ID #I005 [Fixed]

Oracle feed decimals are not validated against the configured price band

Vulnerability Type
Lack of Input Validation (SC05-Lack Of Input Validation)

Severity
Informational

Description
Chainlink feeds report in feed-specific decimals, and the breaker compares the raw price against configured minimum and maximum bounds. The price-range validator only checks that the minimum does not exceed the maximum, not that the bounds match the oracle's decimals. A basis mismatch silently bricks the breaker (always out of range, a permanent pause) or neuters it (too wide to ever trigger). The oracle and the price range are set by independent functions, so swapping to a feed with different decimals never revalidates the range, which is a realistic operational footgun on an oracle migration.

Affected Code

- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F47#L116
- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F46#L192

Impacts
A price band configured in the wrong decimal basis, or an oracle migration to a different-decimal feed, can permanently pause the vault or silently disable depeg protection.

Remediation
Validate the price bounds against the oracle's reported decimals, set the oracle and its range in a single atomic operation, and reject a non-positive minimum bound.

Retest
Fixed. _validateOracle now reads the feed's decimals()

Bug ID #I006 [Fixed]

Heterogeneous-decimal assets on one share corrupt the shared pool and enable a cross-vault drain

Vulnerability Type
Business Logic (SC02-LogicErrors)

Severity
Informational

Description
The shared pool sums raw token amounts from every vault into one accounting total, and the per-vault decimals offset only feeds a negligible virtual-share term, not a normalization of amounts to a common basis. So two vaults holding assets with different decimals have their raw amounts treated as equal in value. The initialization guard only checks that an asset's decimals do not exceed the share's decimals it does not require equal decimals so a lower-decimal asset is accepted and creates a decimal spread. A spread of d decimals is a 10^d mispricing: depositing the higher-decimal asset and redeeming the lower-decimal vault extracts roughly 10^d times the value, while the lower-decimal asset's depositors are correspondingly under-credited at deposit. All live vaults are currently six-decimal, so the spread is zero and the issue is latent; it activates the moment any non-six-decimal asset is onboarded, and would then be a high-impact drain.

Affected Code

- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F39#L65
- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F40#L220

Impacts
Onboarding any asset whose decimals differ from the share's enables a large cross-vault drain and robs that asset's depositors; latent under the current all-six-decimal configuration.

Remediation
Require every vault bound to a share to use exactly the share's decimals, or normalize all amounts to a fixed basis in both the accounting and the conversion math before summing them.

Retest
Initialization now requires exact decimal equality between asset and share.

Bug ID #I007 [Fixed]

Direct asset donations are permanently stuck

Vulnerability Type
Locked Funds

Severity
Informational

Description
tokens transferred directly to a vault inflate its token balance but are never reconciled into the internal idle and total accounting, so they mint no shares and cannot be withdrawn by anyone; there is no skim or sweep function. Besides permanently locking any mis-sent funds, such donations also widen the liquidity-cap over-reporting issue, because the caps read the raw token balance.

Affected Code

- 0x77ea87319f48c584f0c444659d397adc2278101c#code#F46#L266

Impacts
Any tokens accidentally sent to a vault are permanently locked, and they enlarge the over-reporting of redeemable and deployable liquidity.

Remediation
Add a privileged sweep function that can recover the balance in excess of pending deposits, idle balance, claimable-redeem reserves, and accumulated fees.

Retest
Fixed. assetTotalDonated() now nets balances and reserves across all linked vaults before clamping: it sums each vault's physical balance (assetTotalInVault()) and reserved (assetReserved()) separately and returns max(Σbalance − Σreserved − idle, 0), rather than summing the per-vault floored deployable.

6. The Disclosure ----------

The Reports provided by CredShields are not an endorsement or condemnation of any specific project or team and do not guarantee the security of any specific project. The contents of this report are not intended to be used to make decisions about buying or selling tokens, products, services, or any other assets and should not be interpreted as such.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. CredShields does not provide any warranty or representation about the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business. The report is not intended to be used as investment advice and should not be relied upon as such.

CredShields Audit team is not responsible for any decisions or actions taken by any third party based on the report.

Your Secure Future Starts Here

CRED SHIELDS

At CredShields, we're more than just auditors. We're your strategic partner in ensuring a secure Web3 future. Our commitment to your success extends beyond the report, offering ongoing support and guidance to protect your digital assets

AICPA SOC
aicpa.org/soc4so

SCS Advocates
OWASP
OWASP Smart Contract Top 10
Security Risks & Vulnerabilities

20A TANJONG PAGAR ROAD, SINGAPORE (088443)

info@credshields.com